

Inheritance and Polymorphism in Python

Object-Oriented Programming

Through a Geometry Example

Abdesselam Filali

infos@filali.net

github.com/Abdou-fi/Python-OOP

May 30, 2026

Abstract

Python is a multi-paradigm programming language that supports object-oriented programming (OOP) as well as procedural and functional programming. OOP is a programming paradigm that uses objects and classes to structure code in a way that is more intuitive and easier to manage. This document introduces the core OOP concepts of **inheritance** and **polymorphism** through a concrete geometry example, building a class hierarchy of shapes (parallelogram, rectangle, square) using Python and Matplotlib.

Contents

1	What is OOP?	3
2	Advantages of OOP	3
3	Classes and Objects	3
3.1	The <code>__init__()</code> Method	4
3.2	The <code>self</code> Parameter	4
3.3	Properties and Methods	4
3.4	Inheritance and Polymorphism	5
3.4.1	Inheritance	5
3.4.2	Polymorphism	5
4	Geometry Example: Inheritance & Polymorphism	6
4.1	Geometry of the Parallelogram	6
4.2	The <code>Shape</code> and <code>Parallelogram</code> Classes	7
4.3	The <code>Rectangle</code> and <code>Square</code> Classes	10
5	Summary	13

1. What is OOP?

OOP stands for **Object-Oriented Programming**. Python is a fully object-oriented language, allowing you to structure your code using *classes* and *objects* for better organization and reusability.

2. Advantages of OOP

- Provides a clear structure to programs.
- Makes code easier to maintain, reuse, and debug.
- Helps keep your code **DRY** (*Don't Repeat Yourself*).
- Allows you to build reusable applications with less code.

DRY Principle

The DRY principle means you should avoid writing the same code more than once. Move repeated code into functions or classes and reuse it instead.

3. Classes and Objects

Classes and objects are the two core concepts in OOP.

A **class** defines what an object should look like, and an **object** is created based on that class.

In other words, a class is a blueprint for creating objects (a particular data structure), while an object is an *instance* of a class.

Class	Objects
Fruit	Apple, Banana, Mango
Car	Volvo, Audi, Toyota

When you create an object from a class, it inherits all the variables and methods defined inside that class. The main OOP concepts covered in this document are:

1. Classes and objects
2. The `__init__()` method
3. The `self` parameter
4. Properties and methods
5. Inheritance and polymorphism
6. Encapsulation and inner classes (*covered in a future article*)

3.1. The `__init__()` Method

The `__init__()` method is a special method in Python classes, known as the *constructor*. It is automatically called when a new object is created from a class, allowing you to initialize the object's attributes.

```
1 class Person:
2     def __init__(self, name, age):
3         self.name = name
4         self.age = age
5
6 p1 = Person("Amine", 36)
```

3.2. The `self` Parameter

The `self` parameter is a reference to the *current instance* of the class and is used to access variables that belong to it. It does not have to be named `self`, you can call it anything, but it must be the **first parameter** of any method in the class.

```
1 class Person:
2     def __init__(self, name, age):
3         self.name = name
4         self.age = age
5
6     def greet(self):
7         print("Hello, my name is " + self.name)
8
9 p1 = Person("Amine", 36)
10 p1.greet()
```

3.3. Properties and Methods

A **property** is a special type of attribute that lets you define a method as if it were an attribute — you access it without parentheses. A **method** is a regular function defined inside a class that operates on the object.

```
1 class Person:
2     def __init__(self, name, age):
3         self.name = name           # a property or attribute:
4                                     # accessed without parentheses
5         self.age = age
6
7     def greet(self):               # method: called with parentheses
8         return "Hello, my name is " + self.name
9
10 p1 = Person("Amine", 36)
11 print(p1.greet())                # Calling the method using the object
12 print(p1.name)                   # Accessing the property directly
```

3.4. Inheritance and Polymorphism

3.4.1. Inheritance

Inheritance, a powerful feature in object-oriented programming, is the concept of creating a new class that is a modified version of an existing class. The new class inherits all the attributes and methods of the existing class, and you can add new attributes and methods or override existing ones.

Inheritance allows a new class (called a *child* or *subclass*) to inherit attributes and methods from an existing class (called a *parent* or *superclass*). This promotes code reusability and establishes a natural hierarchical relationship between classes.

```
1 class Person:
2     def __init__(self, name, age):
3         self.name = name
4         self.age = age
5
6     def informations(self):
7         print("Hello, my name is " + self.name)
8
9 class Student(Person):
10    def __init__(self, name, age, grade):
11        super().__init__(name, age) # call the parent constructor
12        self.grade = grade
13
14    def informations(self):           # override the parent method
15        print("Hello, my name is " + self.name +
16              " and I am in grade " + str(self.grade))
17
18 p1 = Student("Amine", 36, 12)
19 p1.informations()
```

3.4.2. Polymorphism

Polymorphism allows a unified interface to work with different types of objects. The same method name performs different tasks depending on the object calling it.

```
1 class Person:
2     def __init__(self, name, age):
3         self.name = name
4         self.age = age
5
6     def myfunc(self):
7         print("Hello, my name is " + self.name)
8
9 class Student(Person):
10    def __init__(self, name, age, grade):
11        super().__init__(name, age)
12        self.grade = grade
13
14    def myfunc(self):                 # polymorphic override
15        print("Hello, my name is " + self.name +
16              " and I am in grade " + str(self.grade))
17
```

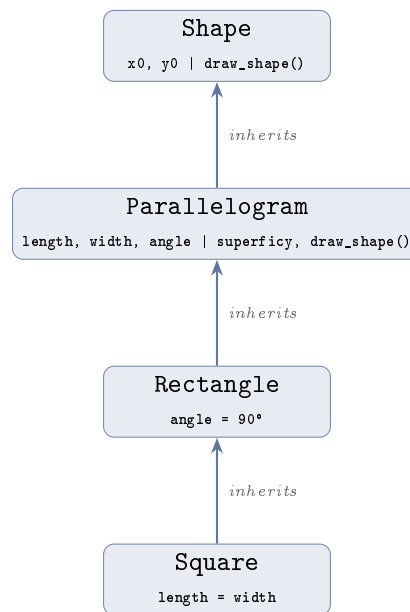
```

18 p1 = Person("Amine", 36)
19 p2 = Student("Sarah", 12, 10)
20
21 p1.myfunc()    # calls Person.myfunc()
22 p2.myfunc()    # calls Student.myfunc()

```

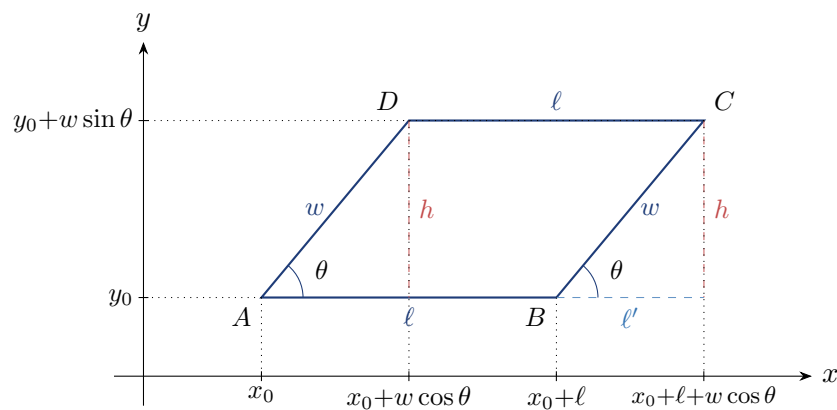
4. Geometry Example: Inheritance & Polymorphism

We now illustrate these concepts through a class hierarchy of geometric shapes. The hierarchy is:



4.1. Geometry of the Parallelogram

The following figure shows a parallelogram with sides ℓ (base) and w (slant side) at angle θ , with height h .



Trigonometric relations:

$$\cos \theta = \frac{\ell'}{w} \implies \ell' = w \cos \theta$$

$$\sin \theta = \frac{h}{w} \implies h = w \sin \theta$$

Vertex coordinates:

$$A = (x_0, y_0)$$

$$B = (x_0 + \ell, y_0)$$

$$C = (x_0 + \ell + w \cos \theta, y_0 + w \sin \theta)$$

$$D = (x_0 + w \cos \theta, y_0 + w \sin \theta)$$

The parallelogram is fully defined by $(x_0, y_0, \ell, w, \theta)$.

4.2. The Shape and Parallelogram Classes

We first define a base class `Shape` and then derive `Parallelogram` from it.

```

1 import matplotlib.pyplot as plt # Import the necessary library
   for plotting
2 from math import cos, sin, pi   # Import mathematical functions
3
4 class Shape:
5     def __init__(self, x0: float, y0: float):
6         self.x0 = x0 # Initialize the x-coordinate of the shape's
           reference point
7         self.y0 = y0 # Initialize the y-coordinate of the shape's
           reference point
8
9     def draw_shape(self): # Define a method to draw the shape,
10        pass # which will be overridden in subclasses
11
12 # Define a class for parallelograms that inherits from the Shape
   class,
13 # since a parallelogram is a specific type of shape with additional
   # properties and methods.
14
15 class Parallelogram(Shape):
16     def __init__(self, x0: float, y0: float,
17                  length: float, width: float, angle: float):
18         super().__init__(x0, y0) # Call the constructor of the
           parent class
19         # Initialize the angle of the parallelogram in radians
20         self.angle = angle
21         # Initialize the length of the parallelogram
22         self.length = length
23         # Initialize the width of the parallelogram
24         self.width = width
25         # Calculate the height using the sine of the angle
26         self.height = width * sin(angle)
27         # Calculate the area using the formula: area = base *
           height
28         self.superficy = length * self.height

```

```

29         # Initialize the coordinates of point A (the reference
           point)
30         self.point_A_x, self.point_A_y = x0, y0
31         # Calculate the coordinates of point B by adding the length
32         # to the x-coordinate of point A
33         self.point_B_x, self.point_B_y = x0 + length, y0
34         # Calculate the coordinates of point C by adding the
           length and
35         # the horizontal component of the width to x0, and the
           vertical
36         # component of the width to y0
37         self.point_C_x = x0 + length + width * cos(angle)
38         self.point_C_y = y0 + width * sin(angle)
39         # Calculate the coordinates of point D by adding the
           horizontal
40         # component of the width to x0, and the vertical component
           to y0
41         self.point_D_x = x0 + width * cos(angle)
42         self.point_D_y = y0 + width * sin(angle)
43
44         # Define a method to print the description of the
           parallelogram,
45         # including its properties: length, width, and the starting
           angle
46         # (converted to degrees for better readability)
47         def description(self):
48             print(f"This is a {self.__class__.__name__.lower()} with
                 the following properties:")
49             print("Length: \t", self.length)
50             print("Width: \t\t", self.width)
51             # Convert angle from radians to degrees for a more
                 intuitive understanding
52             print("Angle (in degrees): \t", self.angle * 180 / pi)
53
54         # Define the x-coordinates of the vertices of the
           parallelogram in order
55         # (A, B, C, D, and back to A to close the shape)
56         def draw_shape(self):
57             data_x = [self.point_A_x, self.point_B_x,
58                       self.point_C_x, self.point_D_x, self.point_A_x]
59             data_y = [self.point_A_y, self.point_B_y,
60                       self.point_C_y, self.point_D_y, self.point_A_y]
61             labels = ['A', 'B', 'C', 'D'] # Define labels for the
                 vertices
62             for i, label in enumerate(labels):
63                 plt.text(data_x[i], data_y[i], label, fontsize=10,
64                           ha='right', va='bottom') # Annotate each
                 vertex
65             plt.plot(data_x, data_y, linestyle='--') # Plot the
                 parallelogram
66             plt.axis('equal') # Equal scaling for x and y axes
67             plt.title(f"{self.__class__.__name__.lower()}
                 representation")
68             plt.show() # Display the plot
69             plt.close() # Close the plot to free resources
70

```

```

71     # Define a method to print the area (superficy) of the
       parallelogram,
72     # which is calculated in the constructor
73     def print_superficy(self):
74         print(f"The superficy of the
              {self.__class__.__name__.lower()} is:
              {self.superficy}")

```

Instantiating and using Parallelogram:

```

1  parallelogram1 = Parallelogram(2, 2, 6, 3, pi/4)
2  parallelogram1.description()
3  parallelogram1.print_superficy()
4  print(f'the superficy of the
        {parallelogram1.__class__.__name__.lower()} is:
        {parallelogram1.superficy}')
5  parallelogram1.draw_shape()

```

This is a parallelogram with the following properties:

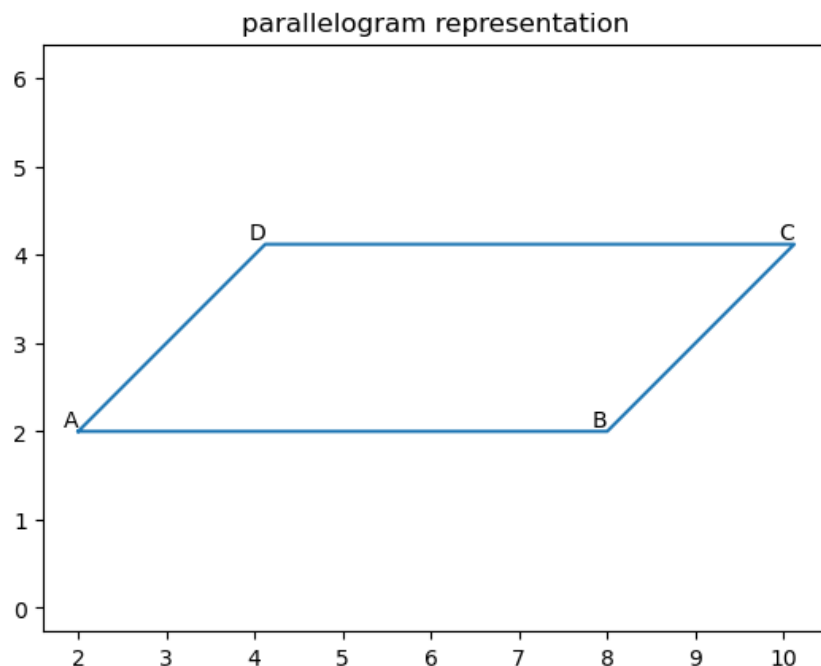
Length: 6

Width: 3

Angle (in degrees):45.0

The superficy of the parallelogram is: 12.727922061357857

the superficy of the parallelogram is: 12.727922061357857



How it works

The **Shape** base class defines the interface (**draw_shape**) without any implementation. **Parallelogram** inherits from **Shape** and overrides **draw_shape** with a concrete Matplotlib-based drawing. This is both *inheritance* (code reuse through the class hierarchy) and *polymorphism* (same method name, different behaviour per class).

4.3. The Rectangle and Square Classes

A rectangle is a parallelogram with $\theta = 90^\circ$, and a square is a special rectangle with equal sides. Both inherit from the class above them in the hierarchy.

```

1  # Define a class for rectangles that inherits from the
    parallelogram class,
2  # since a rectangle is a special case of a parallelogram where the
    angle is
3  # always 90 degrees (pi/2 radians).
4  class Rectangle(Parallelogram):
5      def __init__(self, x0: float, y0: float,
6                  length: float, width: float):
7          super().__init__(x0, y0, length, width, pi/2)
8
9      # Define a method to print the description of the rectangle,
        including
10     # its properties: length and width
11     # (we can omit the angle since it's always 90 degrees for a
        rectangle)
12     # We can also call the description method of the parent class
        to reuse
13     # the code for printing length and width, but we override it
        here to
14     # provide a more specific description for rectangles.
15     def description(self):
16         print(f"This is a {self.__class__.__name__.lower()} with
            the following properties:")
17         print("Length: \t", self.length)
18         print("Width: \t\t", self.width)
19
20     # Define a class for squares that inherits from the Rectangle
        class,
21     # since a square is a special case of a rectangle where length and
        width
22     # are equal.
23     class Square(Rectangle):
24         def __init__(self, x0: float, y0: float, side: float):
25             super().__init__(x0, y0, side, side)
26             self.side = side # Initialize the side length of the
                square,
27                               # which is equal to both length and width
28
29         # Define a method to print the description of the square,
            including
30         # its properties: side length.
31         # We override the parent's description to provide a more
            specific
32         # description for squares.
33         def description(self):
34             print(f"This is a {self.__class__.__name__.lower()} with
                the following properties:")
35             print("Side: \t", self.side)

```

Now we demonstrate all three classes together:

```

1  # Create an instance of the parallelogram class and call its
    methods to
2  # demonstrate functionality
3  # (x0=2, y0=2, length=9, width=4, angle=pi/6 : 30 degrees)
4  parallelogram1 = Parallelogram(2, 2, 9, 4, pi/6)
5  parallelogram1.description()
6  print('the superficy of the parallelogram is:',
        parallelogram1.superficy)
7  parallelogram1.draw_shape()
8
9  # Create an instance of the Rectangle class and call its methods to
10 # demonstrate functionality
11 rectangle1 = Rectangle(1, 1, 10, 4)
12 rectangle1.description()
13 rectangle1.print_superficy()
14 print('the superficy of the rectangle is:', rectangle1.superficy)
15 rectangle1.draw_shape()
16
17 # Create an instance of the Square class and call its methods to
18 # demonstrate functionality
19 square1 = Square(3, 3, 3)
20 square1.description()
21 square1.print_superficy()
22 print('the superficy of the square is:', square1.superficy)
23 square1.draw_shape()

```

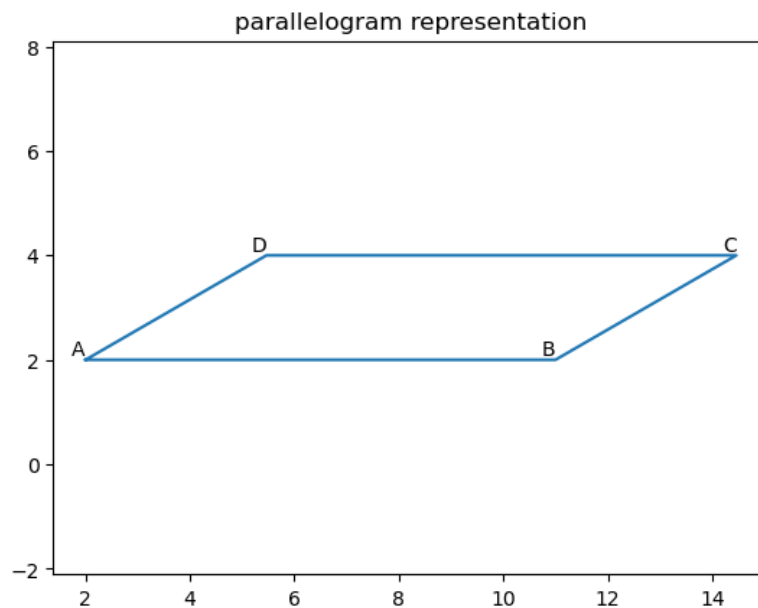
This is a parallelogram with the following properties:

Length: 9

Width: 4

Angle (in degrees): 29.999999999999996

the superficy of the parallelogram is: 17.999999999999996



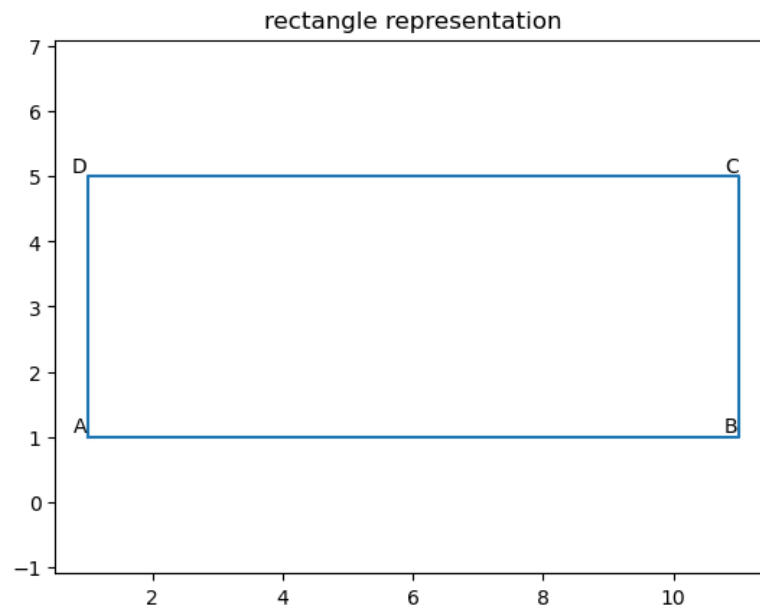
This is a rectangle with the following properties:

Length: 10

Width: 4

The superficy of the rectangle is: 40.0

the superficy of the rectangle is: 40.0

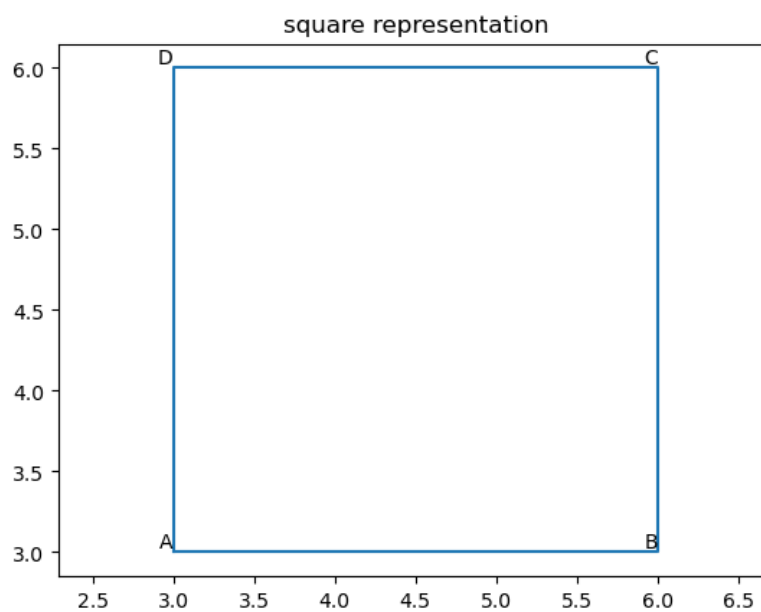


This is a square with the following properties:

Side: 3

The superficy of the square is: 9.0

the superficy of the square is: 9.0



5. Summary

1. We import `matplotlib.pyplot` for plotting and `math` for trigonometric functions.
2. `Shape` defines the base interface with an `__init__` method for coordinates and a placeholder `draw_shape` method.
3. `Parallelogram` inherits from `Shape` and extends it with `length`, `width`, `angle`, `height`, and `area`. It overrides `draw_shape` to render the shape with Matplotlib.
4. `Rectangle` inherits from `Parallelogram`, fixing the angle at 90° and overriding `description`.
5. `Square` inherits from `Rectangle`, constraining `length = width` and overriding `description`.
6. **Polymorphism** is demonstrated through `description()` — the same call produces different output for each class — and through `print_superficy()` and `draw_shape()`, which are inherited and shared across the entire hierarchy.

Key Takeaway

The `super()` call is the mechanism that chains constructors up the hierarchy. Each subclass calls its parent's `__init__` to reuse shared initialisation logic, adding only the parameters it needs — a clean expression of the DRY principle.

Source code available at: github.com/Abdou-fi/Python-OOP *Encapsulation and inner classes will be covered in a future article.*